TECHNICAL WALKTHROUGH — CURRENT CODE

Full Flow: Mint, Treasury, Burn & Payout

Exactly what happens, step by step, from a sender's payment to a recipient's payout — as the code runs today

Mint | Treasury (real vs. dormant) | Burn | Payout

Company: NivixPe Private Limited

Scope: Current (legacy) bridge-service implementation — not the Phase 2 target design

Date: August 2026

FOR INTERNAL / TECHNICAL DISCUSSION

Contents

1. The Core Idea, in One Sentence

> **The token is not currency, and the "treasury" is not one thing.** The token minted and burned in the middle of a transfer is just a temporary receipt — it exists for a few seconds and represents nothing once destroyed. Separately, there are **two different things called "treasury"** in this system, and only one of them holds real money. This document walks through the whole transaction so both points are unmistakable.

Stage	What happens	Real money involved?
1. Mint	Backend creates brand-new tokens from nothing, places them in the sender's wallet	No — the tokens are fabricated
2. Treasury checkpoint	An on-chain "treasury" module exists to track balances of these tokens — but it is not used in a real transaction	No — this step is effectively skipped
3. Burn	The user's own wallet destroys those tokens	No — same fabricated tokens, now gone
4. Payout	Nivix pays the recipient from a real, pre-funded balance sitting inside its payment gateway account	**Yes — this is where real money actually moves**

Sections 3–6 walk through each stage in detail; Section 7 puts all twelve steps into one continuous diagram.

2. Who's Involved

Party	Role
Sender	Pays real fiat money (e.g. INR) into the system
Payment gateway (Razorpay / Cashfree — collection)	Collects the sender's payment into Nivix's bank account; confirms via webhook. INR only.
Nivix backend (bridge-service)	Calculates token amounts, mints, verifies burns, triggers payout
On-chain treasury wallet	A Solana keypair with unlimited mint authority over 7 currency-named tokens; also tracked by a dormant balance/threshold module (Section 4)
User's Solana wallet (Phantom, etc.)	Receives minted tokens; later signs the burn
Solana network	Records mint and burn as public transactions (~2 seconds each)
Cashfree/RazorpayX payout balance	A **real fiat balance** Nivix pre-funds by ordinary bank transfer — this is what actually pays recipients
Recipient	Receives real fiat money out of the system

3. Part One — Mint (Fiat In → Token Created)

1. **Sender pays.** They enter an amount (e.g. ₹8,300) and confirm.
   SENDER · INSTANT

2. **Payment gateway collects it.** Razorpay/Cashfree takes ₹8,300 into Nivix's own bank account. INR only — hardcoded.
   RAZORPAY / CASHFREE · INSTANT

3. **Webhook confirms payment.** Gateway notifies Nivix's backend: "₹8,300 received."
   PAYMENT GATEWAY → BACKEND · SECONDS

4. **Backend calculates token amount.** Using an exchange rate (e.g. ₹83 = 1 unit), it decides: mint 100.
   NIVIX BACKEND · INSTANT

5. **Backend mints.** A `MintTo` instruction creates 100 new tokens directly in the user's wallet, signed by the treasury's private key — the only key allowed to create units of this token.
   NIVIX BACKEND + TREASURY KEY · ~2 SECONDS

6. **Confirmed on-chain.** Total supply of that token permanently increases by 100. Nothing backs this on-chain — trust is entirely internal bookkeeping.
   SOLANA NETWORK · ~2 SECONDS

State after step 6: ₹8,300 sits in Nivix's bank account (real). 100 tokens sit in the user's wallet (fabricated). Nothing has reached a recipient.

4. Part Two — The Treasury Checkpoint

This is the step that causes the most confusion, because the word "treasury" refers to two unrelated things in this codebase.

4.1 Two things, same name

	On-chain "treasury"	Real fiat balance
What it is	A Solana wallet + 7 token accounts, tracked by a module (`treasury-manager.js`) with balance thresholds and an auto-rebalance design	Nivix's actual balance sitting inside its Cashfree / RazorpayX payout account
Holds real money?	No	**Yes** — real rupees, put there by ordinary bank transfer, ahead of time
Used in a real transaction?	**No — dormant**	**Yes — this is what pays the recipient**

Straight from the running code: the moment a payout succeeds through the payment gateway, the backend logs and executes exactly this:

```
"💡 Skipping treasury manager — payout already completed via provider"
```

The balance-check, threshold, and auto-rebalance logic in `treasury-manager.js` is never invoked on this path. It is real, correctly-written code — it is simply disconnected from every transaction that has actually happened.

4.2 Even if it were called, it couldn't do much yet

The configuration file meant to hold this module's thresholds and corridor routing (`treasury-config.json`) is effectively empty — no thresholds, no corridors defined. So even the fallback path that *would* call this module would immediately fail with "No configuration found," rather than actually rebalance anything.

4.3 What this means for the transaction in progress

In the live flow, this checkpoint is a no-op. Nothing changes here — no balance check happens, no threshold is evaluated, no rebalance is triggered. The 100 tokens simply continue sitting in the user's wallet, unaffected, until the next step.

5. Part Three — Burn (Token Destroyed)

7 **User initiates withdrawal.** They choose a recipient and confirm. Their own wallet signs a `Burn` instruction for the 100 tokens.

USER'S OWN WALLET · INSTANT

8 **Burn confirms on-chain.** The 100 tokens are permanently destroyed. Total supply drops back down by exactly the amount minted in step 6.

SOLANA NETWORK · ~2 SECONDS

9 **Backend verifies the burn.** The frontend sends the burn's transaction signature to the backend, which checks Solana directly to confirm it really happened.

NIVIX BACKEND · SECONDS

Why the user signs this, not the backend: the SPL Token standard requires the token account's owner to authorize destroying their own balance. Only the treasury can *create* tokens; only the holder can *destroy* their own. That split is a property of the token standard itself, not a Nivix design choice.

6. Part Four — Payout (Real Fiat Out)

10 **Backend calls the payout provider directly.** Once the burn is verified, it calls Cashfree: "pay ₹8,282 (after fees) to this recipient's bank account." This draws down the **real fiat balance** from Section 4.1 — not anything on-chain.

NIVIX BACKEND → CASHFREE · SECONDS

11 **Recipient is paid.** Cashfree either wires it directly to a bank account, or sends a claim-link the recipient redeems to a bank account or UPI.

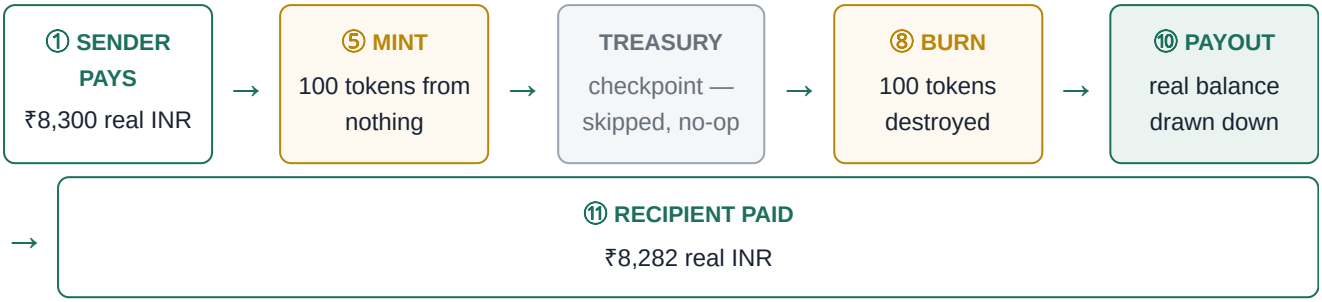
CASHFREE · MINUTES–HOURS

12 **Confirmation shown.** Both sender and recipient see a completed transaction with a receipt in the app.

NIVIX APP · INSTANT

The uncomfortable part, stated plainly: this real fiat balance is refilled manually by Nivix via ordinary bank transfer and is not tracked anywhere in this codebase. Nothing here checks it before a payout, warns if it's getting low, or reconciles it against how many tokens have been minted. If it ran dry, the payout call would simply fail at step 10 with no earlier warning.

7. The Full Flow, Start to Finish




```
₹8,300 IN      100 tokens      "TREASURY"        100 tokens      REAL CASHFREE      ₹8,282 OUT
(real money)   MINTED          CHECKPOINT        BURNED          BALANCE DRAWN      (real money)
          ──►   (fabricated,  ──►  (dormant,  ──►  (destroyed,  ──►  DOWN        ──►
                backed by          skipped —         supply            (the ONLY
                nothing)           no balance        drops)            real-money
                                   check runs)                         step)

Timeline:  Payment (instant) → webhook (sec) → mint (~2s) → [treasury: no-op]
           → user withdraws → burn (~2s) → verify (sec) → payout (min–hrs)
```

Step	Stage	Real money moves?
1–2	Sender pays, gateway collects	Yes — into Nivix's collection account
3–6	Mint	No — fabricated tokens only
Checkpoint	Treasury (dormant)	No — never invoked
7–9	Burn	No — same fabricated tokens, now destroyed
10–12	Payout	**Yes — the real Cashfree balance, drawn down**

8. Today vs. Target — Avoiding the Mix-Up

An earlier investor-facing document describes a **different, not-yet-built system** — a real USDC treasury that mint/burn actually draws from and refills, with genuine threshold-based rebalancing. That is the Phase 2 target design. It is easy to read the two documents side by side and think they contradict each other; they don't — they describe different points in time.

	Today (this document)	Target (Phase 2, not started)
Token	7 self-minted tokens, no real backing	Real Circle USDC
Treasury	Two disconnected things sharing one name (Section 4.1) — only one holds real money, and it's untracked in code	One real, on-chain USDC pool — the actual source and destination of every transfer
Rebalancing	Logic exists in code but is never invoked; config is empty	Genuinely automatic — threshold-triggered top-ups, wired into every payout
Reserve proof	None — trust is internal bookkeeping	Publicly attested 1:1 reserves (Circle), audited monthly

> **Same shape, real asset:** the 12-step flow in Section 7 doesn't change conceptually under the target design. What changes is step 5 ("mint from nothing" becomes "release already-funded, real USDC") and the treasury checkpoint (from a disconnected no-op to the actual, tracked source of the payout). Steps 1–2 and 10–12 — the fiat legs — stay exactly as they are; they just connect to one trustworthy asset instead of a dormant on-chain ledger and an untracked bank balance.

9. Gaps Worth Knowing About

Gap	Detail
Two "treasuries," one real	The on-chain treasury module and the real Cashfree balance are entirely separate; only the second one actually moves money, and it's the one with no tracking or alerting
Real balance is untracked	The fiat balance that funds every payout is refilled manually and checked nowhere in code — no low-balance warning exists before a payout attempt fails
Dormant rebalance logic	`treasury-manager.js` implements threshold checks and auto-rebalancing correctly, but is never called by the real payout path, and its config file is empty regardless
Single point of trust for minting	One private key has unlimited mint authority over all 7 tokens — compromise of that key means unlimited fabricated tokens
No reserve proof	Nothing on-chain shows that minted tokens ever corresponded to real fiat received
Payout and mint/burn are only loosely linked	The burn is verified before payout is triggered, but the amount reconciliation between "tokens burned" and "fiat paid out" happens in application logic, not by any enforced on-chain or ledger constraint

Appendix — Glossary

SPL Token

Solana's standard for creating custom tokens — anyone can create one; whoever holds the "mint authority" key can create new units of it.

Mint authority

The private key authorized to create new units of a token. Here, one treasury key holds mint authority over all 7 currency-named tokens.

Mint (verb)

Creating new tokens and assigning them to a wallet. Increases total supply.

Burn

Permanently destroying tokens from a wallet. Decreases total supply. Must be authorized by the token account's owner.

On-chain treasury (this system)

A Solana wallet and set of token accounts tracked by dormant balance/threshold code — not the source of any real payout today.

Real fiat balance

The actual money sitting in Nivix's Cashfree/RazorpayX account, manually pre-funded, which funds every real payout — untracked in this codebase.

USDC

A real, regulated stablecoin issued by Circle, redeemable 1:1 for US dollars — the asset intended to replace the fabricated tokens and the untracked fiat balance alike, in the target design.